Embeddings and Semantic Retrieval

pig7selene

September 5, 2026

Abstract

Dense retrieval maps a query and a retrieval unit into a shared or aligned vector space, then ranks units by a declared similarity function. This chapter develops that abstraction from Dual-Encoder architectures through vector geometry, normalization, and contrastive training. It explains why independently computed document embeddings make semantic search practical, why negative examples determine the learned ranking boundary, and how dimension, domain, chunk granularity, and storage constrain a deployed retriever. The result is a precise interface between the RAG pipeline introduced in Chapter 32 and the vector-search systems developed next.

1 Introduction

Chapter 32 treated retrieval as an abstract operation that assigns a score $s(q, d_i)$ to a query q and a retrieval unit d_i. That abstraction deliberately admitted lexical, sparse, dense, and hybrid systems. This chapter makes the **dense** case explicit. Its central motivation is simple: useful evidence need not repeat the words used in a query. A query about an "application crash after a configuration change" may be answered by a passage about "a process terminating after an updated setting," even when their literal term overlap is weak. Conversely, overlapping keywords can identify a passage that discusses the right words but the wrong relationship.

Dense retrieval addresses this gap by learning vector representations whose relative geometry is useful for a retrieval task. It does not make language into a space with objectively correct distances. A similarity score remains a model-dependent ranking signal, and semantic similarity is not identical to factual relevance, source authority, or answerability. The chapter therefore focuses on the interface that a dense retriever provides: what is encoded, how scores are constructed, how the representation is trained, and which assumptions must be preserved when the corpus and index evolve.

The word **embedding** already appeared in Chapter 1, but the objects are different. A token embedding is a trainable row associated with one vocabulary ID inside a language model. A retrieval embedding is usually one learned vector for a larger unit–a query, sentence, passage, chunk, or document–whose role is to support comparison with other units. A retrieval encoder may itself be a Transformer, but its output is an externally stored search representation rather than the decoder's token-by-token residual stream.

2 Dense Retrieval as a Ranking Interface

Let the corpus be $D = \{d_1, \ldots, d_N\}$, using the retrieval-unit convention from Chapter 32. A dense retriever applies an encoder to the query and another encoder to a document or chunk:

$$e_q = f_{q(q)} \in R^r, \quad e_d = f_{d(d)} \in R^r. \tag{1}$$

Here r is the embedding dimension. The encoders f_q and f_d may be identical functions, functions with shared parameters, or separately parameterized networks. They must nevertheless produce vectors in a compatible space. A retrieval score is then

$$s(q, d) = \mathrm{sim}\big(e_q, e_d\big), \tag{2}$$

and the system returns the highest-scoring members of D, subject to the filters and tie rules established in Chapter 32. The score is useful primarily for **ordering**. It is not normally a calibrated probability that a passage is true, sufficient, or safe to place in a prompt.

This differs from lexical matching in what the score can exploit. Lexical systems rely on observable terms and their statistics; dense systems compare learned representations that can align paraphrases, related terminology, or patterns learned from supervision. Neither property is absolute. A dense encoder can miss a rare identifier that lexical matching preserves, while a lexical system can retrieve a precise quotation that a broad semantic representation blurs. This chapter does not derive lexical scoring or hybrid retrieval; the point is only that dense retrieval supplies a different evidence signal, not a replacement for every other signal.

3 Dual Encoders and Offline Reuse

A **Dual-Encoder**, also called a **Bi-Encoder**, computes e_q and e_d independently. This independence is the systems property that makes corpus-scale dense retrieval practical. Before requests arrive, the system can encode every retained d_i once, associate its vector with the unit ID and metadata, and build an index. At query time, it computes only e_q and searches among the stored vectors. Dense Passage Retrieval (DPR) made this form particularly influential for open-domain question answering: a question encoder and a passage encoder map their respective inputs into a space scored by an inner product [1].

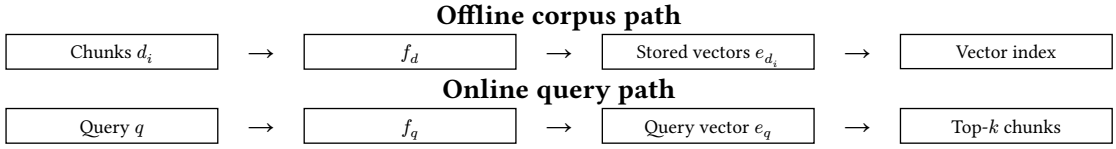

Figure 1: A Dual-Encoder moves corpus work offline. The index stores document-side vectors and identifiers; online retrieval encodes only the new query before ranking compatible stored vectors.

A shared encoder uses one parameter set for both paths, $f_q = f_d$. This is natural for symmetric tasks such as comparing two sentences drawn from similar distributions. Separate encoders allow the model to specialize when queries are short, interrogative, or underspecified while documents are long, declarative, and rich in context. Many search tasks are therefore **asymmetric**: the question "what fixes this?" and a technical explanation of a remediation need not be encoded by exactly the same function. Sharing versus separating parameters is a modeling choice, not a naming convention; the index must record which encoder revision produced every stored vector.

Independent encoding also explains a boundary that later chapters will revisit. A Cross-Encoder may jointly read a query and a candidate passage and can model fine-grained token interactions, but it cannot precompute one candidate representation that is valid for every query. Applying it to every member of a large corpus would discard the offline-reuse property in Figure 1. This chapter does not develop reranking; it establishes why retrieval and later, more expensive comparison stages have different computational roles. Sentence-BERT similarly motivated Siamese encoding so that sentence representations can be compared efficiently rather than requiring a joint model evaluation for each candidate pair [2].

4 Embedding Geometry and Similarity

The geometry of the embedding space is meaningful only together with the scoring rule used in training and retrieval. Three common choices illustrate the distinction. The inner product, or Dot Product, is

$$s_{\mathrm{dot}(q,d)} = e_q^T e_d. \tag{3}$$

It depends on both directions and magnitudes. Cosine Similarity divides out the Euclidean lengths:

$$s_{\cos(q,d)} = \cos\big(e_q, e_d\big) = \frac{e_q^T e_d}{\|e_q\|_2 \, \|e_d\|_2}. \tag{4}$$

Euclidean distance instead measures separation,

$$\delta_{\mathrm{L2}(q,d)} = \|e_q - e_d\|_2, \tag{5}$$

and a system ranks smaller distances ahead of larger ones. These forms are related in important special cases but are not interchangeable by default. A model trained to use inner products may intentionally encode useful confidence or frequency information in vector norm. Replacing its score with Cosine Similarity silently changes its ranking function.

L2 normalization transforms a nonzero vector into

$$\hat{e} = \frac{e}{\|e\|_2}, \quad \|\hat{e}\|_2 = 1. \tag{6}$$

For normalized query and document vectors, Dot Product and Cosine Similarity agree:

$$\hat{e}_q^T \hat{e}_d = \cos\big(e_q, e_d\big). \tag{7}$$

Moreover, squared Euclidean distance becomes $\|\hat{e}_q - \hat{e}_d\|_2^2 = 2 - 2\hat{e}_q^T\hat{e}_d$. Thus, on the unit sphere, maximizing Dot Product, maximizing Cosine Similarity, and minimizing squared Euclidean distance give the same ordering. This equivalence is conditional on normalizing both sides and using exact arithmetic. The appropriate similarity function is the one declared by the embedding model's training and serving contract.

Choice	Ranking quantity	What the score retains
Dot Product	$e_q^T e_d$	Direction and vector magnitude.
Cosine Similarity	$e_q^T \frac{e_d}{\|e_q\|_2 \|e_d\|_2}$	Angular alignment after removing magnitude.
Euclidean distance	$\|e_q - e_d\|_2$	Geometric separation; lower is preferred.
Normalized inner product	$\hat{e}_q^T \hat{e}_d$	The same ordering as cosine and squared L2 distance on unit vectors.

Table 1: Similarity functions encode different assumptions. Normalization creates useful equivalences, but it can also remove magnitude information that a particular encoder learned to use.

5 Contrastive Training and Negative Examples

A retrieval encoder is not useful merely because it produces a vector. Training must state which query–document relationships should be close and which should be separated. Let (q, d^+) be a positive query–document pair and let $\mathcal{B}(q)$ be a candidate set containing d^+ and negatives d^-. A common contrastive objective is

$$\ell_{\mathrm{ctr}(q,d^+)} = -\log \frac{\exp\left(\frac{s(q,d^+)}{\tau}\right)}{\sum_{d_j \in \mathcal{B}(q)} \exp\left(\frac{s(q,d_j)}{\tau}\right)}, \tag{8}$$

where $\tau > 0$ is a temperature parameter. The loss is small when the positive receives a high score relative to all candidates in the denominator. It is not directly a retrieval metric: it is a differentiable training surrogate whose candidate distribution determines what distinctions the encoder is asked to learn. The scoring function in Equation 8 should match the function used to build the later index, including any normalization convention.

This denominator-based comparison is a retrieval-specific instance of contrastive representation learning: the model must identify the positive association among competing candidates rather than assign an independently meaningful score to one vector alone [3]. The task definition determines which candidates compete and therefore which invariances the resulting space learns.

Negative examples are therefore central rather than incidental. **Random negatives** are often easy: an unrelated passage may already score far below the positive and contribute little gradient. **In-batch negatives** reuse the positive documents paired with other queries in the same batch as negatives for the current query. If a batch contains B aligned pairs, each query can compare against up to $B - 1$ additional document vectors without separately constructing that many examples. This makes larger batches valuable for contrastive retrieval, but it also makes the effective negative set and its sampling

policy part of the objective. DPR uses in-batch comparisons and studies both random and BM25-derived hard negatives in its retrieval training procedure [1].

A **hard negative** looks plausible under a lexical or preliminary semantic signal but is not the desired evidence for the query. It can force the model to distinguish near misses that random negatives ignore. Yet hard-negative mining is not infallible. A passage marked negative may be a second valid answer, a useful complementary source, or a different chunk from the same document that supports the query. Such **false negatives** teach the encoder to separate evidence that the application later needs together. Retrieval labels, corpus versioning, and the definition of relevance must consequently be reviewed together rather than treated as independent dataset fields.

6 Semantic Retrieval Pipelines and Storage

The dense pipeline specializes the offline and online paths from Chapter 32. Offline processing parses source documents, creates declared retrieval units, applies f_d, stores one vector per retained unit, and builds a search structure. At query time, the service applies the compatible query tokenizer and f_q, searches the stored vectors by the declared score, and returns the top-k candidate IDs with their scores and metadata. Chapter 34 will examine how a vector-search system can avoid scanning every vector exactly; that systems choice does not change the dense-retrieval interface in Equation 2.

The base storage cost is already instructive. For N stored vectors, dimension r, and b bytes per component, the payload is approximately

$$M_{\text{emb}} \approx Nrb. \tag{9}$$

This excludes IDs, metadata, alignment, replicas, and index overhead. For example, increasing dimension or retaining more chunks increases both memory capacity and the bytes that later similarity kernels must read. More dimensions can offer a model more representational capacity, but they do not guarantee better retrieval; the gain depends on training data, architecture, objective, and the task's required distinctions. Dimension should be selected with quality, storage, bandwidth, and index behavior measured together.

Batch embedding makes offline encoding efficient by applying f_d to many units at once, but it must preserve the unit identity attached to every output row. Long documents make this identity decision consequential. One vector for an entire report can blur a precise claim into a broad average, while smaller passages can expose local evidence to the retriever. The right chunk boundary is a separate design problem, but the representation contract must name which chunk text, title, and metadata fields were encoded. A vector produced from a stale source revision is not repaired merely because its ID still exists.

7 Domain Adaptation and Failure Modes

Embedding quality depends on the distribution that shaped f_q and f_d. A model trained largely on general web question answering may not distinguish the terminology, citation conventions, or relevance criteria of scientific literature, source code, legal documents, or multilingual corpora. Domain adaptation can change the encoder or its training pairs so that positive and difficult-negative relationships reflect the intended corpus. It should be evaluated against held-out queries and source revisions, not inferred from an attractive visualization of a few vectors.

Several failures follow directly from the abstraction. A dense retriever can return a passage that is semantically similar but factually wrong, outdated, or unauthorized; similarity does not establish truth or provenance. An ambiguous query can have multiple valid intents, but one vector may collapse them into a single ranking. Rare entities, identifiers, negation, numerical conditions, and cross-language phrasing can be weakly represented. Poor negatives can produce a shallow ranking boundary, while false negatives can actively suppress useful evidence. A model may also exhibit representation collapse, where many inputs become insufficiently distinguishable, or lose fine-grained detail when a long source is compressed into one vector.

Source changes create a final operational failure mode. Updating raw text without recomputing affected embeddings leaves the vector index semantically stale. Changing the encoder is broader still: query vectors and stored document vectors must be generated by compatible revisions. Mixing incompatible spaces can yield numerical scores while destroying their intended ranking semantics. A published index should therefore identify its corpus revision, chunking policy, document encoder, query encoder, dimension, normalization rule, and score convention.

8 Implementation Contracts

The **encoder contract** must name the query and document encoder revisions, tokenizer and text-normalization rules, maximum input lengths, pooling or readout method, embedding dimension r, output dtype, and normalization policy. If weights are shared, that fact should be explicit; if they are separate, their compatibility must be tested. Given a fixed input and revision, the implementation should produce an embedding with the declared shape and finite components.

The **training contract** must define the positive-pair provenance, candidate set in Equation 8, negative-sampling and hard-negative-mining policies, temperature τ, score function, batch formation, optimizer configuration, and evaluation split. It must state how likely false negatives are detected, filtered, or analyzed. Reporting only one contrastive loss cannot establish retrieval quality; recall-oriented retrieval metrics and error slices must be evaluated on a protected query set.

The **index contract** must bind every vector to a retrieval-unit ID, source and chunk revision, document-encoder revision, dimension, dtype, normalization state, score convention, metadata, and index publication revision. A query service must reject or rebuild incompatible vector sets rather than silently comparing different dimensions or encoder spaces. Tests should verify that offline and online preprocessing agree, L2-normalized vectors satisfy the stated tolerance when required, score ordering matches a high-precision reference on a small corpus, and an updated or deleted source cannot return an obsolete vector after its index revision is published.

9 Summary

Dense retrieval represents a query and a retrieval unit by compatible vectors, then ranks units through a declared similarity function. Dual Encoders make the corpus side reusable: document vectors can be computed offline, while a new query requires only one online encoding and a vector search. The geometry matters only with the model's score convention. L2 normalization makes Dot Product, Cosine Similarity, and squared Euclidean distance equivalent for ranking on the unit sphere, but it can remove magnitude information that an unnormalized model uses.

Contrastive learning turns relevance pairs and negative examples into a ranking boundary. In-batch and hard negatives can make that boundary more informative, while false negatives, domain mismatch, and stale vectors can undermine it. The next chapter takes the stored-vector interface as given and studies how vector-search systems retrieve high-scoring candidates efficiently at corpus scale.

References

[1] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, Association for Computational Linguistics, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[3] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," *arXiv preprint arXiv:1807.03748*, 2018, [Online]. Available: https://arxiv.org/abs/1807.03748